SISTEMA DE AEROPUERTO

Guía de Aprendizaje: Interfaz Gráfica con JFrame

Java Swing | JFrame / JPanel | POO

Este documento explica cada parte nueva que se agregó al proyecto original para convertirlo de una interfaz por consola (*JOptionPane*) a una interfaz gráfica completa con **JFrame**. Además incluye una sección de mejoras propuestas con código listo para aplicar.

1. Estructura General de la Ventana

La clase **AeropuertoApp** ahora extiende **JFrame** en lugar de ser una clase con método *main* simple. Esto la convierte en la ventana principal. El layout principal es **BorderLayout**, que divide la ventana en 5 zonas.

AeropuertoApp.java — constructor

```
public class AeropuertoApp extends JFrame {
// La clase ES la ventana — hereda todo de JFrame
public AeropuertoApp() {
setTitle('Sistema de Aeropuerto');
setSize(900, 650);
setLayout(new BorderLayout()); // divide en 5 zonas
add(crearHeader(), BorderLayout.NORTH); // arriba
add(crearPanelIzquierdo(), BorderLayout.WEST); // menú lateral
add(crearPanelDerecho(), BorderLayout.CENTER); // contenido
add(crearFooter(), BorderLayout.SOUTH); // abajo
}
}
```

¿Por qué extends JFrame?
Al heredar JFrame la clase tiene acceso directo a todos sus métodos (setTitle, setSize, add, setVisible, etc.) sin necesidad de crear un objeto JFrame por separado. Es el patrón estándar para ventanas principales en Java Swing.

Las 5 zonas del BorderLayout

Distribución visual

```
NORTH → Header (título + indicador de estado)
WEST → Panel izquierdo (menú con botones)
CENTER → Panel derecho (contenido dinámico / tarjetas)
SOUTH → Footer (versión y nombre del sistema)
EAST → (no usada en este proyecto)
```

2. Sistema de Colores con Constantes

En lugar de escribir colores directamente en cada componente, se definieron constantes **static final** al inicio de la clase. Esto permite cambiar el tema completo modificando solo un lugar.

Sistema de colores

```
// Constantes de color — se definen UNA sola vez
private static final Color COLOR_FONDO = new Color(15, 23, 42);
private static final Color COLOR_PANEL = new Color(30, 41, 59);
private static final Color COLOR_ACENTO = new Color(56, 189, 248); // azul claro
private static final Color COLOR_TEXTO = new Color(248, 250, 252);
private static final Color COLOR_EXITO = new Color(52, 211, 153); // verde
private static final Color COLOR_ADVERTENCIA = new Color(251, 191, 36); // amarillo
private static final Color COLOR_PELIGRO = new Color(248, 113, 113); // rojo
// Uso en componentes:
panel.setBackground(COLOR_FONDO);
label.setForeground(COLOR_ACENTO);
```

¿Qué significa new Color(R, G, B)?

Java representa colores con valores RGB: tres números del 0 al 255 para Rojo, Verde y Azul. Por ejemplo new Color(56, 189, 248) mezcla poco rojo, bastante verde y mucho azul — dando un azul claro brillante.

3. Panel Izquierdo: Menú con Botones

El menú lateral usa un **BoxLayout vertical**, que apila los componentes de arriba hacia abajo. Cada botón tiene un **MouseListener** para cambiar de color al pasar el mouse (efecto hover).

crearPanelIzquierdo()

```
private JPanel crearPanelIzquierdo() {
JPanel panel = new JPanel();
panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS)); // apila hacia abajo
panel.setPreferredSize(new Dimension(220, 0)); // ancho fijo
agregarBoton(panel, 'Registrar Avion', e -> registrarAvion());
agregarBoton(panel, 'Registrar Vuelo', e -> registrarVuelo());
// ...
return panel;
}
```

Efecto Hover en los botones

MouseAdapter para hover

```
btn.addMouseListener(new MouseAdapter() {
// Se ejecuta cuando el mouse ENTRA al botón
public void mouseEntered(MouseEvent e) {
btn.setBackground(COLOR_PANEL_CLARO); // fondo más claro
btn.setForeground(COLOR_ACENTO); // texto azul
}
// Se ejecuta cuando el mouse SALE del botón
public void mouseExited(MouseEvent e) {
btn.setBackground(new Color(0,0,0,0)); // transparente
btn.setForeground(COLOR_TEXTO); // texto blanco
}
});
```

Lambda vs clase anónima

El código 'e -> registrarAvion()' es una expresión lambda (Java 8+). Es equivalente a escribir new ActionListener() { public void actionPerformed(ActionEvent e) { registrarAvion(); } } pero mucho más corto y legible.

4. Panel Derecho: Contenido Dinámico

El panel derecho no tiene contenido fijo. Cada vez que se presiona un botón del menú, el método **mostrarTarjeta()** limpia el panel y dibuja nueva información. Esto simula la navegación sin cambiar de ventana.

mostrarTarjeta() — actualización dinámica

```
private void mostrarTarjeta(String titulo, String[][] campos, Color colorTitulo) {
panelContenido.removeAll(); // limpia todo el panel
// Agrega el título
JLabel lbl = new JLabel(titulo);
lbl.setForeground(colorTitulo);
panelContenido.add(lbl);
// Agrega cada fila de datos como una 'tarjeta'
for (String[] fila : campos) {
JPanel filaPan = new JPanel(new BorderLayout());
filaPan.add(new JLabel(fila[0]), BorderLayout.WEST); // clave
filaPan.add(new JLabel(fila[1]), BorderLayout.CENTER); // valor
panelContenido.add(filaPan);
}
panelContenido.revalidate(); // recalcula el layout
panelContenido.repaint(); // redibuja en pantalla
}
```

revalidate() y repaint() — el par inseparable

Siempre que se modifica el contenido de un panel en tiempo de ejecución hay que llamar a estos dos métodos. revalidate() recalcula las posiciones y tamaños de todos los componentes. repaint() le dice a Java que vuelva a dibujar el panel en la pantalla. Sin ellos los cambios no se ven.

5. Formularios Multi-campo con JOptionPane

En la versión original cada campo se pedía en un diálogo separado. Con el método **pedirMultiple()** se construye un panel con varios JTextField y se mete dentro de un JOptionPane, logrando un formulario completo en un solo diálogo.

pedirMultiple() — formulario en un diálogo

```
private String[] pedirMultiple(String titulo, String... preguntas) {
// GridLayout crea una cuadrícula de N filas x 2 columnas
JPanel panel = new JPanel(new GridLayout(preguntas.length, 2, 8, 8));
JTextField[] campos = new JTextField[preguntas.length];
for (int i = 0; i < preguntas.length; i++) {
panel.add(new JLabel(preguntas[i] + ':')); // etiqueta
campos[i] = new JTextField(18); // campo de texto
panel.add(campos[i]);
}
// El panel entra DENTRO del JOptionPane
int res = JOptionPane.showConfirmDialog(
this, panel, titulo, JOptionPane.OK_CANCEL_OPTION);
if (res == JOptionPane.OK_OPTION) {
String[] vals = new String[preguntas.length];
for (int i = 0; i < preguntas.length; i++)
vals[i] = campos[i].getText().trim();
return vals; // devuelve todos los valores
}
return null; // usuario canceló
}
```

String... preguntas — varargs

Los tres puntos (varargs) permiten pasar cualquier cantidad de argumentos String. Internamente Java los convierte en un array. Permite llamar al método así: pedirMultiple('Titulo', 'Nombre', 'ID', 'Ciudad') sin crear el array manualmente.

6. Getters Nuevos en las Clases Existentes

La GUI necesita leer datos de los objetos para mostrarlos. Como los atributos son **private**, se agregaron métodos **getter** que devuelven el valor sin permitir modificarlo desde afuera (principio de encapsulamiento).

Getters en Avion, Vuelo y Pasajero

```
// Avion.java — getters nuevos
public String getMatricula() { return matricula; }
public String getModelo() { return modelo; }
public int getCapacidad() { return capacidad; }
// Vuelo.java — getters nuevos
public String getCodigo() { return codigo; }
public String getDestino() { return destino; }
// Pasajero.java — getters nuevos
public String getIdentificacion() { return identificacion; }
public String getNivelSocio() { return nivelSocio; }
```

¿Por qué usar getters en lugar de hacer los atributos public?
Si el atributo fuera public, cualquier parte del código podría cambiar su valor directamente (avion.matricula = 'XYZ'). Con private + getter solo se puede leer. Para modificar habría que crear un setter que podría validar el nuevo valor. Esto es el principio de encapsulamiento de la Programación Orientada a Objetos.

7. Mejoras Propuestas

El proyecto funciona correctamente pero hay varias mejoras que lo harían más robusto, realista y completo. Se listan a continuación con el código correspondiente.

Mejora 1: Múltiples asientos (array en lugar de 3 variables)

Actualmente Avion tiene exactamente 3 asientos con variables separadas (asiento1, asiento2, asiento3). Si la capacidad es 150 eso no escala. La solución es usar un array del tamaño de la capacidad:

Avion.java — asientos con array

```
// ANTES (no escala):
private boolean asiento1, asiento2, asiento3;
// DESPUÉS (escala a cualquier capacidad):
private boolean[] asientos; // array dinámico
public Avion(String matricula, String modelo, int capacidad) {
this.capacidad = capacidad;
this.asientos = new boolean[capacidad]; // todos en false por defecto
}
public void elegirAsiento() {
String input = JOptionPane.showInputDialog('Asiento (1-' + capacidad + '):');
int num = Integer.parseInt(input) - 1; // convertir a índice 0-based
if (num >= 0 && num < capacidad) {
if (!asientos[num]) { asientos[num] = true; }
else { /* ya ocupado */ }
}
}
public double calcularOcupacion() {
int ocupados = 0;
for (boolean a : asientos) if (a) ocupados++;
return (ocupados * 100.0) / capacidad;
}
```

Mejora 2: Múltiples pasajeros (ArrayList)

Actualmente solo se guarda un pasajero a la vez. Con ArrayList se pueden registrar muchos y el manifiesto de comidas mostrará todos correctamente:

ArrayList — múltiples pasajeros

```
import java.util.ArrayList;
// En AeropuertoApp — cambiar:
private Pasajero pasajero = null; // antes: 1 solo
// por:
private ArrayList<Pasajero> pasajeros = new ArrayList<>(); // lista
// Al registrar:
pasajeros.add(new Pasajero(nombre, id, nivel, comida));
// En el manifiesto de comidas:
for (Pasajero p : pasajeros) {
sb.append(p.getNombre()).append(' - ').append(p.getComidaEspecial()).append('\n');
}
```

Mejora 3: Validación de entrada con try-catch

Si el usuario cierra un diálogo sin ingresar nada, parseInt() lanza una excepción y el programa se rompe. Se debe validar siempre:

Validación robusta de enteros

```
// ANTES — se rompe si el usuario cancela o escribe texto:
int cap = Integer.parseInt(JOptionPane.showInputDialog('Capacidad:'));
// DESPUÉS — robusto ante cancelación y texto inválido:
private int pedirEntero(String mensaje) {
while (true) {
String s = JOptionPane.showInputDialog(this, mensaje);
if (s == null) return -1; // usuario canceló
try {
return Integer.parseInt(s.trim());
} catch (NumberFormatException e) {
JOptionPane.showMessageDialog(this,
'Debe ingresar un número entero.', 'Error',
JOptionPane.ERROR_MESSAGE);
}
}
}
// Uso:
int cap = pedirEntero('Ingrese capacidad:');
if (cap < 0) return; // canceló
```

Mejora 4: Tabla de vuelos con JTable interactiva

En lugar de mostrar tarjetas individuales, una JTable permite ver todos los vuelos de golpe, ordenarlos y seleccionar uno con click:

JTable interactiva con ordenamiento

```
String[] columnas = {'#', 'Codigo', 'Destino', 'Salida', 'Llegada', 'Estado'};
Object[][] filas = new Object[contadorVuelos][6];
for (int i = 0; i < contadorVuelos; i++) {
filas[i] = new Object[]{
i+1,
vuelos[i].getCodigo(),
vuelos[i].getDestino(),
horarios[i][0],
horarios[i][1],
horarios[i][2]
};
}
JTable tabla = new JTable(filas, columnas);
tabla.setAutoCreateRowSorter(true); // permite ordenar columnas
tabla.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
// Detectar qué fila se seleccionó:
tabla.getSelectionModel().addListSelectionListener(e -> {
int fila = tabla.getSelectedRow();
if (fila >= 0) System.out.println('Seleccionado vuelo ' + (fila+1));
});
```

Mejora 5: Enum para estados y niveles

Usar Strings libres para estado ('En vuelo', 'En espera', etc.) permite errores de tipeo. Un enum garantiza que solo se usen valores válidos:

Enum EstadoVuelo y NivelSocio

```
// Crear archivo EstadoVuelo.java:
public enum EstadoVuelo {
EN_ESPERA('En espera'),
EN_VUELO('En vuelo'),
ATERRIZADO('Aterrizado');
private final String etiqueta;
EstadoVuelo(String etiqueta) { this.etiqueta = etiqueta; }
public String toString() { return etiqueta; }
}
// Crear archivo NivelSocio.java:
public enum NivelSocio { PLATINO, ORO, REGULAR }
// En Vuelo.java — atributo tipado:
private EstadoVuelo estado; // ya no puede ser cualquier String
// En Pasajero.java:
private NivelSocio nivel;
public double aplicarDescuento(double precio) {
return switch (nivel) { // switch con enum es más limpio
case PLATINO -> precio * 0.90;
case ORO -> precio * 0.95;
case REGULAR -> precio;
};
}
```

Resumen de mejoras

Mejora	Impacto	Dificultad
Array de asientos	Soporta aviones reales	Media
ArrayList de pasajeros	Sistema multiusuario	Baja
Validación con try-catch	No se rompe con inputs malos	Baja
JTable interactiva	UX mucho más profesional	Media
Enum de estados/niveles	Elimina errores de tipeo	Baja

8. Conceptos Clave de Java Swing

JFrame	La ventana principal de la aplicación. Solo debe haber una. Con setDefaultCloseOperation(EXIT_ON_CLOSE) se cierra el programa al cerrar la ventana.
JPanel	Contenedor genérico para agrupar componentes. Se puede anidar dentro de otros JPanels. Tiene su propio layout y fondo.
BorderLayout	Distribuye componentes en 5 zonas: NORTH, SOUTH, EAST, WEST, CENTER. El CENTER toma todo el espacio sobrante.
BoxLayout Y_AXIS	Apila componentes de arriba hacia abajo. Ideal para menús laterales. Se combina con Box.createVerticalStrut(n) para agregar espaciado.
JScrollPane	Envuelve cualquier componente y agrega barras de scroll automáticamente cuando el contenido es más grande que el área visible.
MouseAdapter	Clase abstracta que implementa MouseListener con métodos vacíos. Solo se sobreescribe mouseEntered y mouseExited para el efecto hover.
revalidate() + repaint()	Par obligatorio después de modificar componentes en tiempo de ejecución. revalidate() recalcula el layout, repaint() redibuja en pantalla.
SwingUtilities.invokeLater()	Lanza la GUI en el Event Dispatch Thread (EDT), que es el hilo correcto para Swing. Evita condiciones de carrera en la interfaz.

Resumen del Proyecto

El proyecto pasó de ser un sistema con diálogos uno por uno a una aplicación con interfaz gráfica completa. Los 4 archivos cumplen roles distintos:

Archivo	Rol	Cambios realizados
AeropuertoApp.java	Ventana principal + controlador	Completamente reescrito con JFrame
Avion.java	Modelo del avión	3 getters nuevos agregados
Vuelo.java	Modelo del vuelo	2 getters nuevos agregados
Pasajero.java	Modelo del pasajero	2 getters nuevos agregados

Con las 5 mejoras propuestas el sistema se volvería completamente escalable y listo para un proyecto más avanzado. Se recomienda aplicarlas en el orden listado, empezando por la validación (Mejora 3) que es la más sencilla y más impactante.